TriageGuard

AI-Powered Clinical Triage and Hospital Resource Routing

Technical Documentation

Multi-Hospital Clinical Intelligence & Adaptive Routing Platform

August 2026

1. Overview

TriageGuard is a decision-support system for emergency department triage and bed allocation. It does not diagnose or treat patients — it takes the information already available at triage time and produces two distinct, clearly separated answers for clinical staff:

- **What does this patient clinically need?** — a department preference, driven by a quantitative risk model and historical case reasoning.
- **Where can they actually go right now, at this specific hospital?** — a resource-aware allocation, driven by that hospital's calibrated preferences and its real-time bed state.

The system is built to operate across **multiple independent hospitals** from a single deployment, each with its own configuration, occupancy, historical case data, and calibrated routing behavior.

2. Problem Statement

Emergency departments face two coupled but distinct problems:

- **Clinical**: given incomplete, evolving information, how urgent is this patient, and what level of care do they need?
- **Operational**: given the hospital's actual current capacity, where can that patient safely and immediately be placed?

Static rule-based triage systems typically only address the first problem, and hand-built resource logic is usually hospital-specific and brittle. A useful system must keep these two questions structurally separate — a full ICU does not make a patient less sick, but it does change where they can go — and must scale to more than one facility without duplicating logic per hospital.

3. Solution Overview

TriageGuard answers the clinical question with two complementary models — a compact **XGBoost** risk engine for quantitative, missing-data-aware prediction, and a **RAG + LLM** branch that reasons over the patient's own history and similar historical cases. Their outputs are reconciled into a single confidence-weighted clinical priority.

That clinical priority is handed to an **adaptive routing layer**: a Bayesian policy calibrated from a hospital's own nurses answering a curated set of resource-allocation scenarios, refined where useful with a lightweight PPO-style RL step. This policy, combined with the hospital's live bed state, produces the actual operational recommendation — subject to a hard feasibility gate that never allows an unavailable department to be selected.

Every layer of this pipeline — historical retrieval, calibration data, routing policy, and simulated hospital state — is scoped by **hospital_id**, so Hospital A and Hospital B can be onboarded, configured, and operated independently within the same running system.

4. Key Features

- **Dual clinical models** — calibrated XGBoost risk (ICU risk at 2h/6h/12h, admission risk, confidence) alongside RAG-retrieved historical case reasoning from an LLM.
- **Confidence-weighted reconciliation** — branches blended by how confident each one actually is, not by raw data volume; disagreement can only escalate risk, never suppress it.
- **Nurse-calibrated routing** — a reusable scenario library, automatically filtered and rescaled to each hospital's real departments and bed counts, drives a per-hospital Bayesian policy.
- **RL refinement** — an optional PPO-style policy-gradient step, warm-started from and KL-anchored to the nurse-calibrated policy, trained inside a hospital-scoped simulation.
- **Hard resource feasibility** — a policy preference can never allocate a closed or fully occupied department; a genuine resource conflict is surfaced for human review, never silently overridden.
- **True multi-hospital isolation** — independent configuration, bed state, retrieval, calibration, and routing policy per hospital_id, coexisting in one process.
- **Dynamic hospital simulation** — a simulated ED with arrivals, length-of-stay discharge, and event history, usable per hospital for demos, testing, and RL training.
- **Conversational + dashboard frontend** — a nurse-facing chat agent and a Live Hospital operational view, both hospital-selection aware.

5. System Architecture

Figure 1. End-to-end request flow from nurse interaction to operational recommendation.

6. Clinical Intelligence

XGBoost and the RAG/LLM branch are deliberately complementary rather than redundant:

	XGBoost	RAG + LLM
Question answered	Quantitative risk given current vitals/history	What do this patient's history and similar past cases suggest?
Output	Calibrated icu_risk_2h/6h/12h, admission_risk, predicted_urgency	Structured evidence_strength (1-5), escalation_concern, tr
Missing data	Explicit missingness indicators + information_completeness	Whatever historical evidence retrieval returns
Never does	Reason about longitudinal case history	Output a numeric risk probability or pick the final department

Neither branch is discarded when the other is weak — the reconciler blends both using **each branch's own confidence** (Cx for XGBoost, Cr = evidence_strength / 5 for RAG), so a confident XGBoost prediction is not diluted just because a few vitals are missing, and a branch disagreement or explicit escalation concern can only push the combined risk **up**, never down.

7. RAG Pipeline

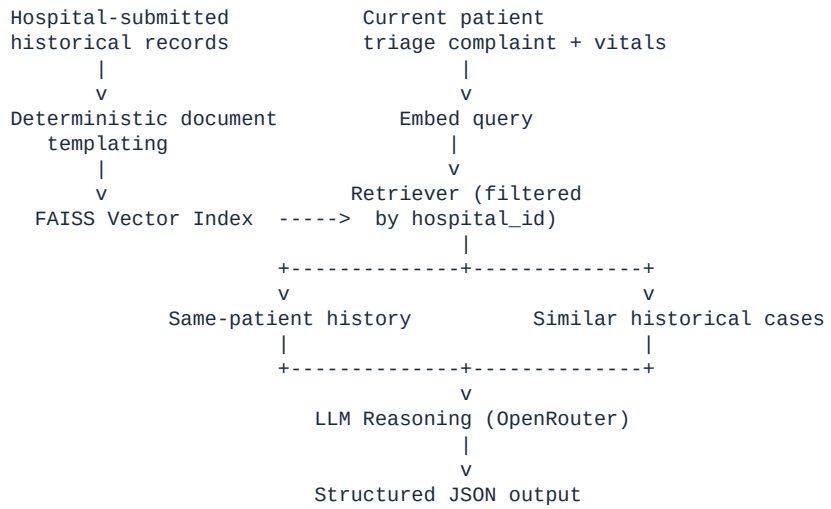

Figure 2. Historical-record ingestion, hospital-scoped retrieval, and LLM reasoning.

- Historical records become documents via deterministic code (never LLM-generated), embedded with a pretrained sentence-transformer into a local FAISS index.
- Retrieval returns a small, bounded number of the patient's own prior visits and similar cases from other patients — never dozens of documents.
- **Hospital isolation**: every document is tagged with its source hospital; retrieval filters on that tag so one hospital's cases can never inform another's reasoning.
- The LLM reasons over retrieved evidence and returns a structured assessment — including a self-rated evidence_strength — but never a numeric probability and never the final department.

8. Adaptive Hospital Routing

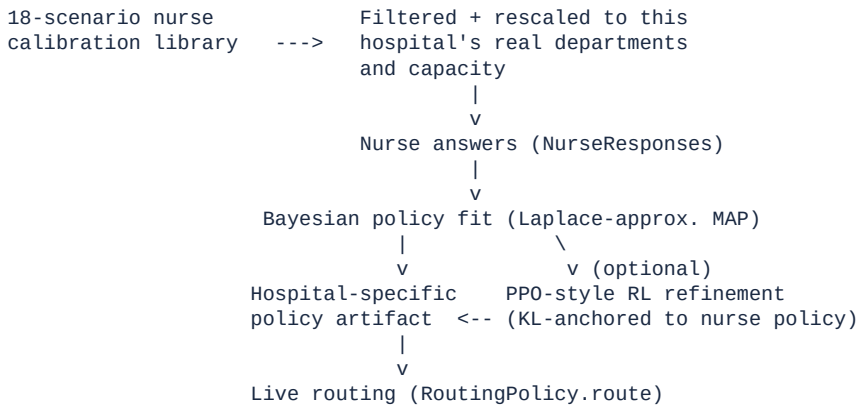

Figure 3. From reusable scenario library to a live, hospital-specific routing policy.

- A single reusable library of 18 hand-designed clinical/resource scenarios covers urgency, model agreement, confidence, and resource-pressure ladders.
- For each hospital, scenarios requiring a department it doesn't have are excluded, and bed counts are rescaled onto its real capacity while preserving each scenario's clinical intent.
- A nurse's answers fit a **Laplace-approximated Bayesian linear policy** — appropriate for the handful of demonstrations available, avoiding overfitting a conventional model.
- An optional **lightweight PPO-style policy-gradient** step refines the policy inside a hospital-scoped simulated environment, initialized from and anchored to the nurse policy.
- A hard safety layer selects the best-scoring *feasible* department, or explicitly reports a resource conflict — a policy can never cause allocation to a closed or full department.

9. Multi-Hospital Architecture

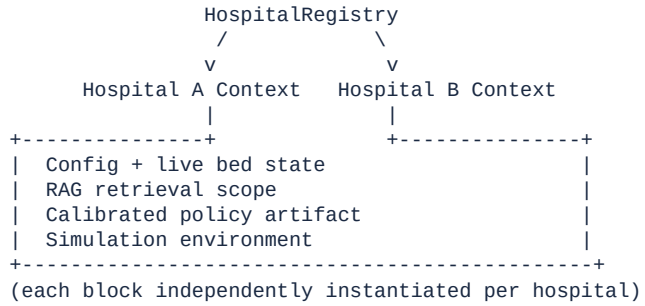

Figure 4. One registry, fully isolated per-hospital stacks.

Layer	Isolation mechanism
Configuration & departments	Per-hospital hospital_config.json; a missing department (e.g. no CICU) excludes scenarios/routing requirin
Live bed state	Dedicated HospitalStateService per hospital; the original deployment is preserved as the reserved "default"
Historical retrieval (RAG)	FAISS retrieval filtered by each document's hospital_id tag.

Layer	Isolation mechanism
Routing policy	Bayesian/RL artifacts saved under data/routing_policy/<hospital_id>/, never a shared global file.
Simulation	Each HospitalSimulator owns its own event engine, patient queue, and bed state.

A request always carries hospital_id from the frontend through the API, the clinical pipeline, the routing policy, and the simulated hospital state — hospital identity is never silently dropped along the way.

10. Dynamic Hospital Simulation

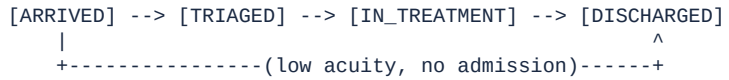

Figure 5. Simulated patient lifecycle.

- Patients arrive (auto-generated per scenario rate, or manually triggered/entered), are triaged through the real clinical pipeline, and occupy a bed for a simulated length of stay before automatic discharge releases it.
- The simulation clock can be stepped manually (controlled demos) or advanced automatically.
- A live event feed (arrivals, capacity warnings, discharges, scenario changes) makes state changes visible rather than opaque.
- Pre-loaded patients and department-grouped, reorderable queue views support a populated, realistic queue for demonstration.
- Preset scenarios (Normal Day, Busy Day, Surge, Resource-Constrained) demonstrate the same hospital under different pressure levels.

11. Frontend / Nurse Experience

- **Hospital Selector** — every session is scoped to one hospital; switching hospitals switches which state, queue, and policy the rest of the UI reflects.
- **Dashboard** — a landing view summarizing hospital load and recent activity.
- **Live Hospital** — department occupancy, the current patient queue (grouped by department, reorderable), and simulation controls.
- **Patient List / Patient Workspace** — per-patient clinical detail, assessment results, and the operational recommendation.
- **Manual Intake Form** — lets staff add a patient into the live queue directly.
- **Chat Dock** — the conversational agent, with a confirmation modal gating any write action behind explicit staff approval.

Throughout, the UI surfaces the **operational** recommendation as the actionable answer, with the clinical preference and any resource constraint or human-review flag available for context — not raw model internals.

12. Backend / API

api_server.py is a FastAPI layer over the existing agent/pipeline code — it does not reimplement clinical or routing logic.

Category	Endpoints
Health / hospitals	GET /api/health, GET /api/hospitals
Conversational agent	POST /api/session, GET /api/session/{id}, POST /api/chat, POST /api/tools/execute, POST /api/tools/confirm
Patients	GET /api/patients, GET /api/patients/{id}, POST /api/patients/{id}/assess
Hospital state	GET /api/hospital/state
Simulation	GET /api/simulation/scenarios, GET/POST /api/simulation/dashboard, /scenario, /step, /arrival, /manual-arrival

hospital_id is accepted throughout this surface and threaded into the clinical pipeline, routing policy, and simulation layer as described in Section 9.

13. End-to-End Workflow

1. A nurse selects **Hospital A** in the frontend and opens the Live Hospital view.
2. A patient arrives (simulated or manually entered) with a chief complaint and available vitals.
3. XGBoost and RAG/LLM run in parallel and are reconciled into a clinical priority and preference.
4. Hospital A's calibrated policy scores candidate departments using Hospital A's own live occupancy.
5. The feasibility gate checks real bed availability, stepping down to the best available alternative or reporting a resource conflict.

6 The nurse sees the operational recommendation and the reasoning behind it.

7 Running the identical patient against **Hospital B** can produce a different operational recommendation, while the clinical assessment stays the same.

14. Safety, Reliability & Fallbacks

- **Resource feasibility is a hard gate** — an unavailable or closed department can never be the final allocation.
- **No fabricated allocation** — when nothing is both clinically acceptable and available, the system reports an explicit resource conflict instead of guessing.
- **Asymmetric escalation** — disagreement or an explicit escalation concern can only raise reconciled clinical risk, never lower it.
- **Policy fallback** — a hospital with no calibrated policy yet falls back to the plain clinical preference automatically.
- **Default-hospital backward compatibility** — the original single-hospital deployment is preserved as the reserved "default" hospital.
- **Human-in-the-loop writes** — every state-changing action requires explicit staff confirmation before it is committed.
- **Validated nurse calibration** — a chosen department is validated against that scenario's actual candidate set before it can influence policy fitting.

TriageGuard is a decision-support prototype. It does not perform autonomous diagnosis, does not replace clinical judgment, and is not presented as validated for production clinical deployment.

15. Technology Stack

Layer	Technology
Clinical risk model	XGBoost, scikit-learn, joblib
Text embeddings	sentence-transformers (MiniLM)
Vector retrieval	FAISS
LLM reasoning & agent	OpenRouter API (model-agnostic), custom tool-calling loop
Routing policy	PyTorch (Bayesian linear policy; PPO-style policy gradient)
Backend API	FastAPI, Uvicorn
Frontend	React 19, TypeScript, Vite, Tailwind CSS, React Router
Testing	pytest
Data formats	JSON-based configuration and artifacts throughout (no database)

16. Testing & Validation

The suite currently totals **324 collected tests** across the clinical pipeline, RAG, routing/policy, agent runtime, hospital registry/isolation, and multi-hospital simulation: **317 passing**, **7 known failing**, none caused by the multi-hospital/routing work described in this document.

- 4 pre-date this work: a tracked test fixture (patients/52.json) was overwritten by a manual interactive session, drifting from the value its own tests expect.
- 3 stem from the merged live-hospital-queue feature seeding a simulator's patient queue with pre-simulated patients by default, which a small number of older tests (predating that feature) assume is empty at construction.

Meaningful coverage specific to this work includes:

- Multi-hospital state, RAG-retrieval, and simulation isolation (mutating one hospital never affects another; identical patient IDs across hospitals never collide).
- Hospital-specific Bayesian/RL policy fitting and artifact storage (different calibration produces different policy weights; artifacts never collide between hospitals).
- Live-routing integration proving the same clinical patient can receive different operational recommendations at two differently-configured hospitals, and that an infeasible department is never surfaced as the final answer.
- Regression coverage confirming default/single-hospital behavior is unchanged when hospital_id is omitted.

No performance or accuracy numbers for the XGBoost/RAG models themselves are claimed beyond what the pipeline structurally guarantees — the underlying models were trained on a small demonstration dataset, not a clinically validated one.

17. Setup & Running

Prerequisites: Python (with a virtual environment), Node.js, and an OpenRouter API key.

```
# 1. Environment
cp .env.example .env
# edit .env and set OPENROUTER_API_KEY

# 2. Backend dependencies (install per module -- no single root manifest yet)
pip install -r triageguard_xgb/requirements.txt
pip install -r triageguard_rag/requirements.txt
pip install fastapi uvicorn torch faiss-cpu python-dotenv httpx

# 3. Run the backend API
python -m uvicorn api_server:app --reload --port 8000

# 4. Frontend
cd frontend
npm install
npm run dev

# Terminal-only conversational agent (no frontend required)
python scripts/chat_with_agent.py
```

The frontend dev server proxies API calls to the backend; open the printed local URL and select a hospital to begin.

18. Limitations & Future Scope

- **RL is not yet part of live routing.** The Bayesian policy drives live, per-hospital routing today; PPO-style RL refinement is implemented and hospital-scoped, but remains a separate, explicitly-invoked training step.
- **No consolidated dependency manifest** — backend dependencies are split across per-module requirements files plus a few ungrouped packages required by the agent/API layers.
- **Simulation scenario capacities are hospital-agnostic** — loading a preset scenario applies its own canonical bed counts to whichever departments a hospital has, rather than rescaling to that hospital's configured capacity the way calibration scenarios already do.
- **XGBoost/RAG models are demonstration-scale**, trained on a small dataset for prototype purposes, not clinically validated.
- **Nurse calibration responses are currently developer-authored placeholders** for the scenario library's defaults; real deployment assumes actual clinical staff answer the set.

19. Conclusion

TriageGuard demonstrates that clinical risk assessment and hospital operational routing can be built as genuinely separable concerns — a quantitative model and a historical-reasoning model produce a clinical picture that never changes with bed availability, while a hospital-specific, nurse-calibrated policy decides allocation under a hard safety gate. That separation, combined with a hospital_id boundary threaded consistently through configuration, state, retrieval, calibration, and routing, lets the same engineering effort serve one hospital or many without duplicating logic — the core requirement for a system meant to generalize

beyond a single deployment.